

Trabalho – Engenharia de Software I

Guilherme Nogueira Zanon

João Lucas Oliveira

Desenvolvimento de um Framework C++ para Construção de Simulações Baseadas na Dinâmica de Sistemas

Questão 1 – Casos de Uso e Critérios de Aceitação

UC-01: Criação de Sistemas

Descrição: Sistemas representam estoques ou acumuladores que armazenam uma quantidade numérica ao longo da simulação (populações, água, energia, recursos financeiros ou qualquer variável acumulativa). O usuário deve criar sistemas informando um nome e um valor inicial.

C/C++

```
System populacao("Populacao", 100.0);  
cout << populacao.getName() << ": " << populacao.getValue();  
// Populacao: 100  
  
populacao.setValue(150.0);  
cout << "Novo valor: " << populacao.getValue();  
// Novo valor: 150
```

Critério de Aceitação:

1. O sistema deve registrar e salvar com sucesso o nome e o valor inicial fornecidos pelo usuário.
2. O nome do registro deve ser mantido e exibido exatamente da forma como foi digitado na criação.
3. Deve ser possível consultar o valor armazenado e alterá-lo a qualquer momento, garantindo que a atualização seja salva com precisão.
4. O sistema deve aceitar o número zero como um valor inicial ou atual válido.

UC-02: Criação de Fluxos

Descrição: Fluxos representam mecanismos de transferência entre sistemas durante a simulação. Cada fluxo possui um sistema de origem e um sistema de destino, sendo responsável por calcular uma quantidade que será transferida. A classe **Flow** é abstrata e define a interface comum para todos os tipos de fluxo

C/C++

```
System populacao("Populacao", 100.0);
System recursos("Recursos", 0.0);

Flow* fluxo = new FlowExponential(&populacao, &recursos);

cout << fluxo->getOrigem()->getNome(); // Saída: Populacao
cout << fluxo->getDestino()->getNome(); // Saída: Recursos
cout << fluxo->execute();               // Saída: 1
```

Critério de aceitação:

1. O sistema deve registrar e salvar com sucesso dois sistemas como origem e destino de um fluxo.
2. A associação entre fluxo e sistemas de origem e destino deve ser mantida e acessível exatamente da forma como foi configurada na criação.
3. Deve ser possível consultar os sistemas de origem e destino de um fluxo a qualquer momento, garantindo que as referências sejam preservadas com precisão.
4. O sistema deve aceitar um ponteiro nulo como origem ou destino, representando uma fonte ou sumidouro externo.

UC-03: Execução do Modelo Exponencial

Descrição: O crescimento populacional ocorre de maneira proporcional à população atual, seguindo a equação $P(t+1)=P(t) + r \cdot P(t)$, onde r é a taxa de crescimento. A população cresce sem limitação de recursos.

```
C/C++
// class ExponentialFlow: public Flow
// ExponentialFlow (System* source, System* sink);

System populacao("Populacao", 10.0);

FlowExponential fluxo(&populacao, &populacao);

cout << fluxo.execute() << endl; // Saida: 3 (30% de 10)
```

Critério de Aceitação:

1. O sistema deve calcular e retornar com sucesso o valor do fluxo exponencial com base no sistema de origem.
2. A taxa de crescimento deve ser aplicada exatamente como definida na equação do fluxo, sem arredondamentos ou desvios.
3. Deve ser possível executar o fluxo repetidamente e obter valores proporcionais ao estado atual do sistema de origem, garantindo que cada execução reflita o valor mais recente.
4. O sistema deve aceitar valores zero ou negativos no sistema de origem como entrada válida para o cálculo.

UC-04: Execução do Modelo Logístico

Descrição: Além da taxa de crescimento, existe uma limitação ambiental $P(\max)$ (capacidade máxima). A equação logística é:

$$P(t + 1) = P(t) + 0.3 P(t) \times \left(1 - \frac{P(t)}{P(\max)}\right)$$

```
C/C++
//LogisticFlow(System* origem, System* destino, double pmax);

System populacao("Populacao", 10.0);
LogisticFlow fluxo(nullptr, &populacao, 70.0);

cout << fluxo.execute(); // Saída: 2.57143 (30% de 10 * (1 - 10/70))
```

Critérios de aceitação:

1. O sistema deve calcular e retornar com sucesso o valor do fluxo logístico considerando a população atual e a capacidade máxima do ambiente.
2. O fator limitante $(1 - P/P_{\max})$ deve ser aplicado exatamente como definido na equação, reduzindo o crescimento à medida que a população se aproxima de $P_{\max}$.
3. Deve ser possível configurar diferentes valores de $P_{\max}$ para cada fluxo logístico, garantindo que cada instância utilize sua própria capacidade máxima.
4. O sistema deve aceitar valores de $P_{\max}$ maiores que zero, e valores populacionais iguais ou superiores a $P_{\max}$ devem resultar em crescimento zero ou negativo.

UC-05: Execução Temporal da Simulação (Model)

Descrição: O Model é o orquestrador da simulação. Ele gerencia os sistemas e fluxos, executando um loop temporal onde todos os fluxos são calculados primeiro e depois aplicados aos sistemas de destino.

```
C/C++
System populacao("Populacao", 100.0);
System recursos("Recursos", 0.0);

FlowExponential fluxo(&populacao, &recursos);

Model modelo;
modelo.add(&populacao);
modelo.add(&recursos);
modelo.add(&fluxo);
modelo.run(0, 10);

cout << populacao.getValor(); // Saída: Populacao reduzida apos 10
iteracoes
cout << recursos.getValor(); // Saída: Recursos aumentados apos 10
iteracoes
```

Critério de Aceitação:

1. O sistema deve executar com sucesso o número exato de iterações especificado entre os tempos inicial e final.
2. Todos os fluxos devem ser calculados primeiro, antes de qualquer sistema ser atualizado, garantindo que a ordem dos fluxos não afete os resultados da iteração atual.
3. O valor calculado de cada fluxo deve ser subtraído do sistema de origem e adicionado ao sistema de destino a cada iteração, garantindo a conservação da transferência.
4. O sistema deve aceitar intervalos de tempo com início igual ao fim (zero iterações), resultando em nenhuma alteração nos sistemas.

Questão 3 – Cenários de teste

A validação dos casos de uso e critérios de aceitação foi realizada com base nos modelos fornecidos no software Vensim. O arquivo de validação disponibilizado no trabalho foi utilizado como referência para análise do comportamento esperado dos modelos exponencial e logístico.

Os resultados produzidos pela API desenvolvida foram comparados com os resultados observados no Vensim, permitindo verificar a coerência matemática e comportamental das simulações. Durante os testes realizados, o modelo exponencial apresentou crescimento contínuo e acelerado, enquanto o modelo logístico apresentou crescimento limitado pela capacidade máxima do ambiente, estabilizando-se próximo ao valor de $P_{\max}$. Além disso, a validação também foi realizada por meio de testes funcionais automatizados utilizando a biblioteca padrão do C++. Esses testes permitiram verificar o correto funcionamento das classes System, Flow, ExponentialFlow, LogisticFlow e Model, garantindo que os critérios de aceitação definidos anteriormente fossem satisfeitos.

Validação do Modelo Exponencial

Parâmetros: $P(0) = 10$, $r = 0,3$, 10 iterações.

Equação: $P(t+1)=P(t) + r \cdot P(t)$

Iteração	API Desenvolvida	Vensim	Diferença
t=0	10,0000	10,0000	0,0000
t=1	13,0000	13,0000	0,0000
t=2	16,9000	16,9000	0,0000
t=3	21,9700	21,9700	0,0000
t=4	28,5610	28,5610	0,0000
t=5	37,1293	37,1293	0,0000
t=6	48,2681	48,2681	0,0000
t=7	62,7485	62,7485	0,0000
t=8	81,5731	81,5731	0,0000
t=9	106,0450	106,0450	0,0000
t=10	137,8585	137,8585	0,0000

Validação do Modelo Logístico

Parâmetros: P0 = 10, r = 0,3, Pmax = 70, 50 iterações.

Equação: $P(t + 1) = P(t) + 0.3 P(t) \times (1 - \frac{P(t)}{P(max)})$

Iteração	API Desenvolvida	Vensim	Diferença
t=0	10,0000	10,0000	0,0000
t=10	60,5000	60,5000	0,0000
t=20	69,8000	69,8000	0,0000
t=30	70,0000	70,0000	0,0000
t=40	70,0000	70,0000	0,0000
t=50	70,0000	70,0000	0,0000

Questão 4 – Projeto UML da API

